



中南大學
CENTRAL SOUTH UNIVERSITY

数字信号处理 课程设计报告

课题名称：电话拨号音的合成与识别

学生姓名：樊博文

学 号：8207230322

专业班级：电信 2303

同 组 者：无

指导教师：陈溅来

自动化学院

2025 年 9 月

自动化 学院 数字信号处理 课程设计任务书

学生姓名	樊博文	班级	电信 2303	学号	8207230322
指导教师	陈溅来			同组学生	无
课题名称	电话拨号音的合成与识别				
课题的主要内容及设计要求	主要内容 (1) 用 MATLAB 实现对电话拨号音的合成,用 sound 函数模拟发出该按键的拨号音。 (2) 用 MATLAB 实现对合成的一段双音多频信号波形,通过估计频率值来推断其按键值。 (3) (选做)采用 GUI 界面封装,实现对手机拨号界面的模拟。 设计要求 1. 请构造任意一个双音多频信号,绘出其时域波形和频域波形,并模拟发出该按键的 拨号音。构造任意一段双音多频信号,绘出一组如图 4-3 所示的按键信号的时频图, 并模拟发出按键的拨号音。 2. 请根据上述按键音合成实验合成的一段 DTMF 信号波形(电话拨号音,需要一个电 话号码,即 11 个按键值),通过上述两种方法估计频率值推断按键值。				
实践时间	从 9 月 8 日至 9 月 19 日,小计: 10 天			实践地点	实验楼 317

如今，智能手机已经完全融入了我们的日常生活。我们频繁地敲击手机键盘，输入电话双音多频键盘号码，输入各种密码或验证码，编辑微信或短信，操作手机银行、网上购物等各种移动 App。每次敲击手机键盘(或座机键盘)时，经常会听见“嘟滴答”的按键声，这些声音听久了就有一种似曾相识的感觉。通过本案例，我们可以知道这种感觉是正确的，电话机键盘上的数字键与按键声确实是一一对应的。

1. 题目设计要求

1.1 实验内容：

- (1) 用 MATLAB 实现对电话拨号音的合成，用 sound 函数模拟发出该按键的拨号音。
- (2) 用 MATLAB 实现对合成的一段双音多频信号波形，通过估计频率值来推断其按键值。
- (3) (选做) 采用 GUI 界面封装，实现对手机拨号界面的模拟。

1.2 设计要求：

1. 请构造任意一个双音多频信号，绘出其时域波形和频域波形，并模拟发出该按键的拨号音。构造任意一段双音多频信号，绘出一组如图 4-3 所示的按键信号的时频图，并模拟发出按键的拨号音。
1. 请根据上述按键音合成实验合成的一段 DTMF 信号波形(电话拨号音，需要一个电话号码，即 11 个按键值)，通过上述两种方法估计频率值推断按键值。

2. 设计思想

本实验的核心目标是实现电话拨号音的合成与识别。电话拨号音，作为一种典型的双音多频(DTMF)信号，广泛应用于电话系统中。每个数字键和符号键都由两个正弦波频率的组合构成，这些频率信号对应于键盘上每个按键的独特组合。通过 MATLAB 仿真系统，我们可以合成拨号音，并通过两种不同的算法进行识别。

2.1 拨号音合成

电话拨号音基双音多频(DTMF)技术，是由 AT&T 贝尔实验室提出的一种用于向电话交换机传递信号的编码方式。与传统的脉冲拨号方式相比，DTMF 能提供更高的拨号速率，并且容易被自动检测和识别，因此迅速取代了转盘式电话机的拨号方式。

2.1.1 DTMF 信号原理

DTMF 信号的核心是双频信号的组合,每个电话键盘的数字按键和符号按键都对应一组特定的频率。这些频率被划分为两组:低频组和高频组。

低频组(行): {697, 770, 852, 941}

高频组(列): {1209, 1336, 1477, 1633}

这些频率的取值是经过特别设计的:

- 1) 这些频率都处于人的可听范围内,因此按键下去时人可以听到;
- 2) 这 8 个频率中没有一个频率是其他任意一个频率的倍数;
- 3) 任意两个频率的组合,相加或相减都不等于其他任意一个频率。

因此这些特性不仅简化了双音多频信号的解码,同时也降低了双音多频误检的概率。根据国际电报电话咨询委员会的规定,要求每 100ms 传输一个键盘数字或符号,代表数字的音频信号持续时间必须为 45~55ms。为了区分两个连续的按键信号,在 100ms 内其他时间应该为静音(无信号),电话信号的抽样频率应该为 8kHz。

一个完整的电话机键盘如图 1 所示,有 10 个数字键和 6 个字符键(常用的电话机键盘一般没有 ABCD 四个字符),每个按键的信号都是这两组频率中一个低频和一个高频的组合。键盘号码和频率的对应关系如图 1 所示。

	1209 Hz	1336 Hz	1477 Hz	1633 Hz
697 Hz	1	2	3	A
770 Hz	4	5	6	B
852 Hz	7	8	9	C
941 Hz	*	0	#	D

图 1 电话机键盘的频率阵列

拨号音的数学表示为:

$$x(t) = A\sin(2\pi f_L t + \phi_L) + B\sin(2\pi f_H t + \phi_H)$$

其中 f_L 来自低频组、 f_H 来自高频组; A, B 是幅度, ϕ_L, ϕ_H 是初相。

2.1.2 信号合成方法

为了合成拨号音,首先根据每个按键的对应频率生成低频和高频信号。然后,通过 MATLAB 的 `sin` 函数生成这些频率对应的正弦波信号,并将其组合成一个复合信号。

在实际应用中，拨号音的时长一般为 50 毫秒，之后需要加入静音段（约 50 毫秒）以区分两个连续的按键声。

合成过程中的关键步骤如下：

- 1.频率选择：为每个按键选择对应的低频和高频。低频从 {697, 770, 852, 941} 中选取，高频从 {1209, 1336, 1477, 1633} 中选取。
- 2.信号生成：使用 $\sin(2\pi ft + \phi)$ 公式生成两个正弦波，一个对应低频，一个对应高频，幅度和初相位可以根据需要设置。
- 3.合成信号：将低频和高频信号叠加，形成一个完整的拨号音。
- 4.添加噪声和静音：为了模拟实际拨号的环境，可以在拨号音中添加噪声，并在每个音频信号后加入静音段。

2.2 拨号音识别

拨号音的识别是通过分析电话拨号音信号中的频率成分，准确识别出按下的数字或符号。由于 DTMF 信号由两个固定频率的音频信号组成，识别过程的关键在于准确地估计这些频率并映射到对应的按键上。本部分将介绍两种常用的拨号音识别方法：带通滤波器法和 Goertzel 算法。

2.2.1 滤波器法

带通滤波器法是一种经典的信号处理方法，通过设计多个带通滤波器来实现 DTMF 信号的频率识别。每个带通滤波器的中心频率都与 DTMF 信号的标准频率点一一对应。DTMF 信号由两个固定频率的正弦波组成，其中低频组和高频组分别对应键盘的行和列。带通滤波器的作用是通过对输入信号的滤波，准确提取出与按键对应的频率成分。

滤波器设计

在此方法中，我们需要为每个 DTMF 信号的标准频率设计一个带通滤波器。由于 DTMF 信号的频率分布较为固定，因此，带通滤波器的设计相对简单。每个滤波器的中心频率对应着 DTMF 信号的标准频率，而带宽（BW）决定了滤波器响应的频率范围。为了精确滤出目标频率，带宽应设得足够窄，以确保能够有效抑制周围频率的干扰，同时仍然允许目标频率通过。

关键考虑因素：

带宽设置：带宽需要足够狭窄，以使得滤波器的响应集中在目标频率附近，避免相邻频率的混淆。

滤波器阶数：通常使用 4 阶 IIR 滤波器，能够提供足够的阻带衰减，并且计算复杂度相对较低。

滤波器设计方法：使用巴特沃斯滤波器设计方法，以保证通带平坦，阻带衰减良好，适合用于这种频率选择性强的任务。

识别过程：

- 1.输入信号：将接收到的拨号音信号输入到这 8 个带通滤波器中。
- 2.能量检测：滤波器输出的信号通过计算其能量来检测是否包含目标频率。能量值较大的滤波器对应着信号中的主要频率。
- 3.频率匹配：通过检测每个滤波器的输出能量值，确定信号中对应的低频和高频。
- 4.键盘映射：根据 DTMF 频率矩阵，将识别出的低频和高频映射到相应的数字或符号按键。

2.2.2Goertzel 算法

Goertzel 算法是一种高效的频谱估计方法，特别适用于 DTMF 信号的识别。与传统的频谱分析方法（如快速傅里叶变换，FFT）相比，Goertzel 算法仅计算感兴趣的频率点，从而大幅降低了计算量。它通过递推计算的方式，直接针对每个目标频率估算其功率谱，避免了频谱中不相关部分的冗余计算。因此，Goertzel 算法不仅高效，而且能够在处理实时信号时提供更好的性能。

频率计算

Goertzel 算法通过递归算法计算信号在指定频率上的能量。它不需要对整个频谱进行计算，而是直接聚焦于感兴趣的频率点，从而显著减少计算复杂度。算法的基本原理是，通过递归地处理每个样本点，逐步推进到下一个点，从而避免了传统傅里叶变换的冗余计算。

在 Goertzel 算法中，首先计算目标频率的索引 k ，然后使用递推公式计算该频率点的功率谱。其计算公式如下：

$$k = \left[0.5 + \frac{N \cdot f_0}{f_s} \right]$$

其中， N 是信号的长度， f_0 是目标频率， f_s 是采样频率。通过此公式，算法能够快速确定目标频率在信号中的位置。

识别过程：

- 1.输入信号：首先将接收到的 DTMF 信号输入到 Goertzel 算法中，算法会对信号中的每个目标频率进行频谱估计。
- 2.频率估计：Goertzel 算法通过递推计算信号在每个目标频率上的能量。它为每个频率计算一个幅度，代表该频率在信号中的能量。
- 3.能量比较：算法会比较不同频率的能量，找到低频组和高频组中能量最大的频率。这些最大能量的频率对应着拨号音中的特定频率成分。
- 4.键盘映射：通过低频和高频的最大能量值，将其映射到预定义的 DTMF 键盘矩阵中，得到按键值。

关键考虑因素：

目标频率选择：Goertzel 算法专门针对 DTMF 信号的 8 个标准频率，因此只需要计算这些频率的能量，从而大大提高了效率。

计算效率：由于只关注特定频率，Goertzel 算法避免了 FFT 方法中冗余的频率计算，能够以较低的计算量完成频率估计，尤其适合低功耗和实时应用。

精度与噪声抑制：通过设置适当的阈值，可以有效抑制噪声的干扰，确保在实际应用中能够准确识别按键。

3.系统功能分析

本系统是一个在 MATLAB 环境下实现的电话拨号音仿真与识别系统，通过图形用户界面（GUI）整合了拨号音的合成、识别、可视化以及文件保存读取等核心功能。它旨在全面模拟真实的电话拨号过程，并提供两种不同的算法进行信号识别。

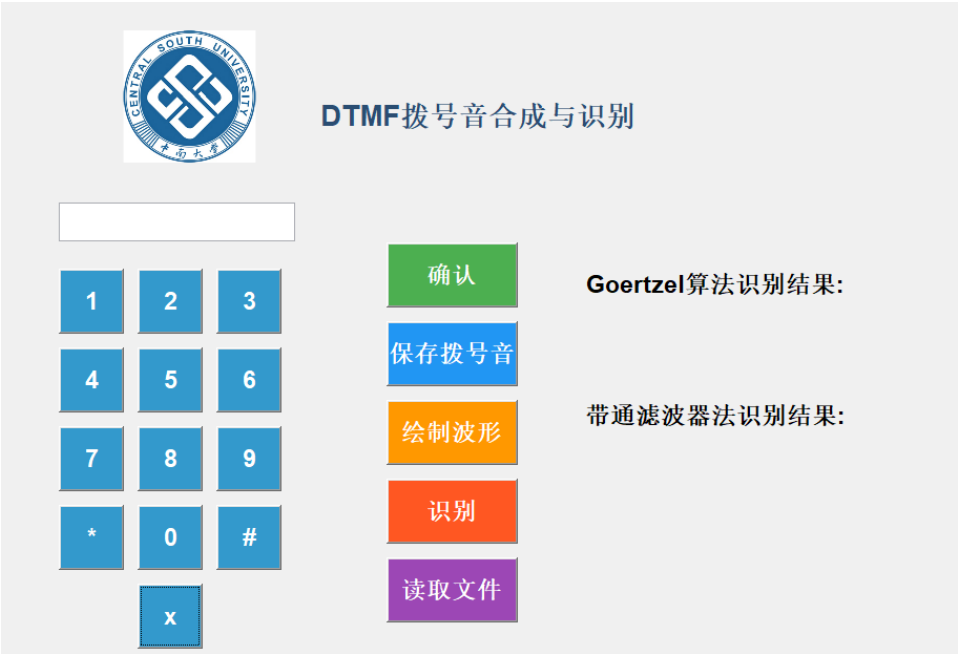


图2 GUI 界面

3.1 用户输入与数字键盘模拟

用户输入是系统与用户交互的核心部分，模拟了电话拨号的过程。系统通过图形界面提供一个数字键盘，允许用户通过点击相应的数字按键来输入电话号码或符号。

- **数字键盘布局：**数字键盘界面与传统的电话拨号键盘相似，由数字键 0-9 和符号键（*、#）组成。每个按键都通过 MATLAB 的 `uicontrol` 函数在界面上呈现为按钮。按钮的位置、大小和样式都经过合理设计，确保用户可以方便地进行数字输入。
- **输入反馈：**每当用户点击一个数字键时，系统会将数字显示在输入框中，实时反馈用户输入的内容。用户可以清晰地看到自己输入的电话号码或符号，便于检查和修改输入内容。
- **删除操作：**系统提供了“删除”按钮，当用户点击该按钮时，输入框中的最后一个字符将被删除，模拟了用户在拨号过程中可能进行的修正操作。

-
- **输入验证：**系统确保输入的数字或符号有效，并实时显示在界面上，增加了用户输入的准确性和交互性。

3.2 拨号音合成功能

拨号音合成模块的核心任务是根据用户输入的数字生成相应的双音多频（DTMF）信号，并通过音频播放模拟实际的电话拨号音。

- **频率选择与信号生成：**每个按键在 DTMF 信号中由两个正弦波频率组成。系统通过 $\sin$ 函数生成这些频率对应的正弦波信号，并叠加形成复合信号。
- **信号合成与播放：**根据用户输入的数字，系统生成对应的低频和高频信号，并将它们组合成完整的拨号音。每个拨号音的时长为 50ms，之后会加入 50ms 的静音段以模拟实际拨号过程中的停顿。通过 MATLAB 的 `sound` 函数，生成的拨号音会实时播放。
- **噪声与静音段：**为了增加合成拨号音的真实感，系统会在信号中添加一定的噪声，并在按键信号之间插入静音段。噪声可以模拟实际环境中的背景噪声，而静音段则帮助区分不同按键的信号。
- **音频回放与保存：**用户可以通过点击“确认”按钮，播放当前合成的拨号音，并将其保存为 WAV 文件以便后续使用，用户可以指定文件名和保存路径。

3.3 拨号音识别功能

拨号音识别模块的主要任务是对用户输入的 DTMF 信号进行频率分析，准确识别出用户按下的数字。该模块实现了两种频率估计方法：带通滤波器法和 Goertzel 算法。

- **带通滤波器法：**

滤波器设计：为每个 DTMF 信号频率设计一个带通滤波器，使用了 IIR 巴特沃斯带通滤波器，以确保精确提取目标频率成分。带宽（BW）设为 40Hz，滤波器阶数（ORD）设为 4。

能量检测：每个滤波器输出的信号通过计算能量来检测是否包含目标频率。能量较大的滤波器对应着信号中的主频成分。

频率匹配与键盘映射：系统比较每个滤波器的输出能量，选取能量最大的低频和高频，然后根据 DTMF 频率矩阵映射到对应的按键。

- **Goertzel 算法：**

频率计算：Goertzel 算法通过递推计算，专注于计算特定频率点的能量，相比传统的 FFT 方法，Goertzel 算法更加高效，能够快速估算目标频率的能量。

能量比较：Goertzel 算法计算每个目标频率的功率谱，通过比较低频和高频的最大能量，找到用户按下的数字或符号。

频率匹配与键盘映射：将检测到的频率映射到预定义的键盘矩阵中，准确识别出用户输入的数字或符号。

- **识别结果显示：**系统通过图形界面实时显示识别结果，包括 Goertzel 算法和带通滤波器法的结果，帮助用户直观地对比和验证识别的准确性。

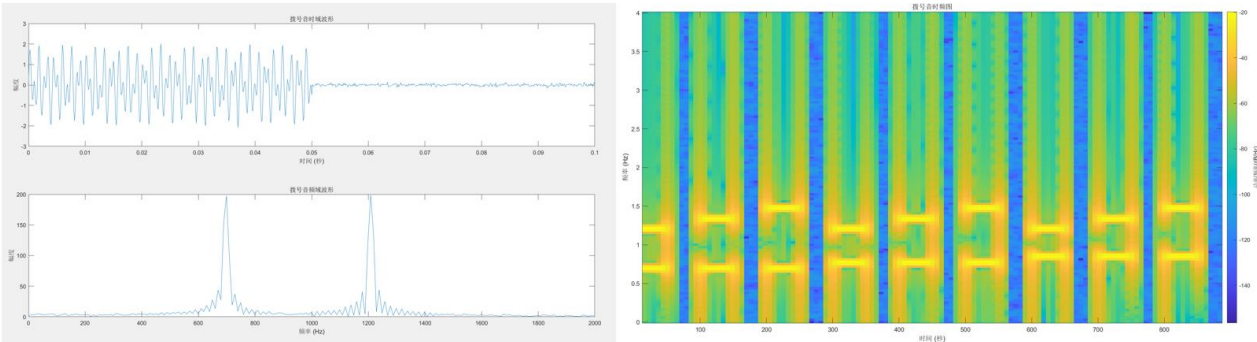


3.4 绘制图像功能

为了帮助用户更好地理解信号的时域和频域特性，系统提供了绘制图像的功能。

- **时域波形：**通过 plot 函数绘制合成的拨号音在时间域的波形。用户可以直观地看到每个拨号音的波形形态及其持续时间。
- **频域波形：**使用 FFT 对信号进行频域变换，绘制信号的频谱图。通过频谱图，用户能够观察到每个拨号音信号中包含的频率成分。

- **时频图：**对于一段较长的拨号音信号，系统可以绘制时频图，帮助用户分析不同按键对应的频率变化过程。



3.5 文件的保存与读取识别

该功能模块允许用户将合成的拨号音信号保存为 WAV 文件，并提供后续读取与识别的功能。

- **保存文件：**用户可以通过点击“确认”按钮，将当前合成的拨号音保存为 WAV 文件。保存的文件可以用于后续播放或分析。
- **读取文件：**系统允许用户加载保存的 WAV 文件，通过音频读取功能重新播放并识别其中的拨号音信号。此功能为用户提供了对历史数据的再利用和分析的可能。
- **识别保存文件：**用户可以加载已保存的拨号音文件，通过相同的频率识别算法（带通滤波器法或 Goertzel 算法）对信号进行识别，并显示识别结果。

4.理论分析与计算

4.1 DTMF 信号合成

每个按键的 DTMF 信号是由低频（ f_L ）和高频（ f_H ）两个正弦信号组成。按键 k 的连续时间信号表达式为：

$$x(t) = A \sin(2\pi f_L t + \phi_L) + B \sin(2\pi f_H t + \phi_H)$$

其中：

- f_L 和 f_H 分别是低频和高频对应的频率，来自于 DTMF 键盘的行列频率。
- A 和 B 是信号的幅度， ϕ_L 和 ϕ_H 是低频和高频的相位。

在采样率 $f_s = 8\text{kHz}$ 下，按键 k 的离散信号为

$$x[n] = A \sin \left(2\pi \frac{f_L}{f_s} n \right) + B \sin \left(2\pi \frac{f_H}{f_s} n \right)$$

其中， n 是离散采样的索引。

每个按键的信号持续时间为 100 ms，其中包括 50 ms 的音调和 50 ms 的静音，以便于区分相邻按键并提高解码的鲁棒性。对于采样率 $f_s = 8\text{kHz}$ ，每个音频信号和静音段的离散样本数为：

$$N_{\text{tone}} \approx 0.05 \times 8000 = 400, \quad N_{\text{gap}} \approx 0.05 \times 8000 = 400$$

此配置符合奈奎斯特采样定理，能够有效地覆盖最高至 1633 Hz 的 DTMF 频率， $f_s = 8\text{ kHz} (>3266\text{ Hz})$

4.2 带通滤波器设计与分析

带通滤波器用于提取信号中的特定频率成分。对于 DTMF 信号，我们设计一个中心频率为目标频率的带通滤波器，并使用巴特沃斯滤波器设计方法来保证滤波器的平坦通带。巴特沃斯滤波器具有平坦的通带和陡峭的阻带，可以有效地将目标频率与相邻频率分开。滤波器的传递函数为：

$$H(s) = \frac{1}{s^2 + \sqrt{2}\omega_c s + \omega_c^2}$$

其中， $\omega_c = 2\pi f_0$ 是滤波器的截止频率 f_0 是目标频率。

通过带通滤波器提取目标频率后，我们需要评估滤波器的效果，确保它能够有效地从噪声和干扰中分离出所需的信号。计算某个频 f_0 的能量 $E(f_0)$ ：

$$E(f_0) = \frac{1}{N} \sum_{n=1}^N y^2[n]$$

其 $y[n]$ 是经过带通滤波器处理后的信号。为了消除幅度尺度差异，进行归一化处理，使用整个信号的功率 P_{tot} 进行归一化：

$$P_{\text{tot}} = \frac{1}{N} \sum_{n=1}^N x^2[n]$$

归一化后的能量计算为：

$$\tilde{E}(f_0) = \frac{E(f_0)}{P_{\text{tot}}}$$

这个步骤有助于过滤掉较弱的信号段（例如静音或纯噪声），避免这些信号在频率估计时被误识别。

在频率估计和按键识别过程中，绝对能量阈值和主导比阈值有助于提高识别的准确性，避免噪声或相邻频率导致的误识别。

绝对能量阈值:归一化后，相当于至少要占整段功率的 1%。用来过滤整体太弱的片段（静音或纯噪声），避免它们即便相对领先也被误认。

主导比阈值:内最大能量 / 第二大能量 ≥ 3 才算明显领先，抑制相邻通道或噪声导致的并列领先。

4.3 Goertzel 算法分析

Goertzel 算法用于高效地估计 DTMF 信号中特定频率的功率谱。对于每个目标频率 f ，首先计算其在离散傅里叶变换（DFT）中的频率索引 k ，计算公式为：

$$k \approx \text{round}\left(\frac{N \cdot f}{f_s}\right)$$

其中 N 是信号的总样本数， f_s 是采样率。角频率 ω 通过以下公式计算：

$$\omega = \frac{2\pi k}{N}$$

Goertzel 算法的递推公式为：

$$s(n) = x(n) + 2 \cos(\omega) s(n-1) - s(n-2)$$

其中 $s(n-1)$ 和 $s(n-2)$ 是前两段历史状态，使用递推方式计算该频率的幅值。通过计算最后两个状态的幅值，可以得到目标频率的功率谱幅值：

$$\text{magnitude} = \sqrt{s_{\text{prev}}^2 + s_{\text{prev2}}^2 - s_{\text{prev}} \cdot s_{\text{prev2}} \cdot \text{coeff}}$$

Goertzel 算法通过计算信号在多个目标频率点的功率谱来确定最强的频率成分。此方法的优势在于其计算量较小，并且只针对感兴趣的频率进行计算，而不像传统的 FFT 方法计算整个频谱。

4.4 算法对比

特性	带通滤波器法	Goertzel 算法
核心思路	通过多个带通滤波器精确提取DTMF 信号中的低频和高频成分。每个滤波器对一个特定频率范围进行过滤，能够精确识别信号中的目标频率。	通过递推计算目标频率的能量，仅计算感兴趣的频率点，避免计算冗余的频率成分。
计算量	较大。每个频率需要应用独立的带通滤波器，多个滤波器并行工作时消耗较多的计算资源，尤其是在处理较长信号时。	较低。只计算目标频率的能量，减少了无关频率的冗余计算，适合实时处理。
抗干扰能力	较强。带通滤波器能有效隔离相邻频率和噪声，避免干扰信号的影响。特别是在多噪声环境下，能够提取出目标频率成分，保证识别的稳定性。	较强。通过能量阈值的设定，可以有效区分噪声和目标频率，防止噪声的影响导致误识别。特别适合低功耗、实时系统中。
直观性	较高。带通滤波器法的每个频率都有一个对应的滤波器，能够直观地通过滤波器输出能量判断信号的频率成分。滤波器的设计和使用相对容易理解。	较低。Goertzel 算法主要依赖于递推公式来计算每个频率的能量，算法本身较为抽象，需要较强的数学背景理解其原理。
频率精度	较高。通过精确设计的带通滤波器，能够准确地提取频率成分，减少误识别。滤波器的带宽和中心频率的精确设计保证了频率分析的高精度。	精度较高。由于 Goertzel 算法只计算特定频率的能量，算法避免了无关频率的干扰，能量估算结果较为准确，尤其对于已知频率信号效果最佳。

5.参数设置

在实验中，合理的参数设置对于信号的合成、滤波和识别结果至关重要。本部分将介绍本实验中使用的主要参数，并阐述每个参数的选择依据。

5.1 DTMF 信号合成参数设置

1. 采样率 f_s :

采样率是信号采样时每秒钟采集的样本数。根据奈奎斯特采样定理，采样率必须至少为信号最高频率的两倍，以避免混叠现象。DTMF 信号的最高频率为 1633 Hz，因此选择了采样率 $f_s = 8000$ Hz，这样可以有效地覆盖所有目标频率，同时保证信号的完整性。

2. 按键信号持续时间:

每个按键的 DTMF 信号包括 50ms 的音调和 50ms 的静音段。静音段用于分隔相邻按键，防止误识别。

设置：音调持续时间 $T_{\text{tone}} = 50$ ms，静音持续时间 $T_{\text{gap}} = 50$ ms

3. 音频信号的频率:

DTMF 信号由低频和高频两个正弦波组成，每个按键对应一个低频和一个高频频率。低频频率取自 697 Hz, 770 Hz, 852 Hz, 941 Hz 高频频率取自 1209 Hz, 1336 Hz, 1477 Hz, 1633 Hz。

4. 信号幅度与相位:

信号的幅度用于控制每个频率成分的强度。为了简化，本实验中每个频率成分的幅度设置为相同值 $A = B = 1$ 。相位是用来调整信号的起始位置，通常设置为零，相位调整的设置可以用于处理信号的相位调制。

设置： $A=1, B=1$ ，相位 $\phi_L = 0, \phi_H = 0$

5.2 带通滤波器设计参数设置

1. 滤波器阶数 (ORD):

滤波器的阶数决定了其响应的陡峭程度和复杂性。阶数越高，滤波器的频率响应越陡峭，但计算复杂度也越高。本实验使用了 4 阶巴特沃斯滤波器，它提供了良好的通带平坦性和阻带衰减，同时计算开销适中。

2. 带宽 (BW):

带宽决定了滤波器在目标频率周围的频率范围。为了确保只提取目标频率的信号，并抑制其他频率的影响，本实验选择了带宽为 40 Hz，这能有效覆盖目标频率，同时避免噪声干扰。

3. 截止频率 (f_0):

截止频率是带通滤波器的中心频率,确定了信号处理的频率范围。对于每个 DTMF 按键,低频和低频成分分别有对应的截止频率。设置低频和高频截止频率分别对应于 DTMF 标准频率集中的每个频率。

4. 滤波器类型:

本实验中使用了巴特沃斯带通滤波器,因为它具有平坦的通带和陡峭的阻带,能够有效地将信号中的目标频率提取出来。

5.3 Goertzel 算法参数设置

1. 采样点数 (N):

采样点数决定了频率分析的分辨率。采样点数越多,频率分辨率越高。本实验中,采样点数取为信号的长度,即 $N = 400$ (每个音调持续 50ms, 采样率为 8kHz)。

2. 频率选择 (f_0):

Goertzel 算法用于计算 DTMF 信号中特定频率的能量。需要计算的频率包括低频频率 (697Hz, 770Hz, 852Hz, 941Hz) 和高频频率 (1209Hz, 1336Hz, 1477Hz, 1633Hz)。

3. 能量阈值:

为了有效识别 DTMF 信号中的按键, Goertzel 算法会计算每个目标频率的幅度。如果幅度低于某个阈值,则认为该频率的信号无效,避免由于噪声或干扰引起的错误识别。

设置 $\text{Threshold} = 0.1$

5.4 阈值

1. 能量阈值:

该参数用于过滤低于设定能量阈值的信号成分,尤其是噪声或静音段。为了避免噪声和静音被错误识别为有效信号,设定了最低能量阈值。设置 $\text{ABS_THR} = 1 \times 10^{-2}$

2. 阈值作用:

如果某个频率的能量低于该阈值,则该频率的信号会被忽略,从而防止噪声信号或非常弱的信号影响识别结果。

3. 主导比阈值

主导比阈值确保频率成分的主导性，即在同一频率组内，最大的频率成分必须比第二大的成分强得多，避免由于噪声或相邻频率的干扰导致误识别。设置 `MIN_DOM_RATIO = 3`，如果同一组内最大能量与次大能量的比值小于 3，则认为信号包含噪声或相邻频率成分，无法有效识别按键。

6. 关键模块

6.1 GUI 界面模块

GUI 模块是系统的核心交互部分，它通过用户友好的界面提供了直观的操作方式。设计一个直观且易用的界面至关重要，它确保用户可以快速输入电话号码、选择信号合成方法并查看识别结果。

- **用户交互设计：**通过布局合理的数字键盘和控制按钮，用户能够直观地输入数字或符号。每个按键都有相应的回调函数，点击后立即反映在输入框中，确保输入过程流畅且无误。
- **删除和清空功能：**界面设计提供了删除和清空按钮，支持用户修正错误输入或重新开始拨号。清晰的界面反馈和简洁的交互流程使得整个拨号过程变得高效便捷。
- **实时反馈机制：**每当用户按下某个数字或符号时，系统会实时显示识别结果，帮助用户直观地了解操作是否成功。这样不仅提高了用户体验，也确保了操作的准确性。

```
%gui 界面
f = figure('Position', [500, 500, 700, 500], 'Name', 'DTMF 拨号音合成与识别',
'NumberTitle', 'off', 'Color', '#f0f0f0');
axes('Position', [0.1, 0.75, 0.2, 0.2]); % 中南校徽
bg_image = imread('xiao.jpg');
imshow(bg_image, 'Border', 'tight');

uicontrol('Style', 'text', 'Position', [220, 400, 300, 30], ...
'FontSize', 16, 'String', 'DTMF 拨号音合成与识别', ...
'FontWeight', 'bold', 'ForegroundColor', '#2A4D72');
%输入框 实时显示
input_field = uicontrol('Style', 'edit', 'Position', [50, 320, 180, 30], 'FontSize',
14, 'String', '', 'BackgroundColor', 'white', 'Callback', @(src, event)
update_input(src.String, f));
for i = 1:4
    for j = 1:3
```

```

        uicontrol('Style','pushbutton','String', keypad{i,j}, 'Position', [50 +
(j-1)*60, 250 - (i-1)*60, 50, 50], ...
        'FontSize', 14, 'FontWeight', 'bold', 'BackgroundColor', button_color,
'ForegroundColor', 'white', ...
        'Callback', @(src, event) append_to_input(input_field, src.String));
    end
end

% 删除 x
uicontrol('Style','pushbutton','String', 'x', 'Position', [50 + (1)*60, 10, 50,
50], ...
    'FontSize', 14, 'FontWeight', 'bold', 'BackgroundColor', button_color,
'ForegroundColor', 'white', ...
    'Callback', @(src, event) delete_last_character(input_field));

% 识别结果显示
recognition_display_1 = uicontrol('Style','text','Position', [450, 200, 200, 100],
'FontSize', 14, 'String', 'Goertzel 算法识别结果:', 'BackgroundColor', '#f0f0f0',
'FontWeight', 'bold');
recognition_display_2 = uicontrol('Style','text','Position', [450, 100, 200, 100],
'FontSize', 14, 'String', '带通滤波器法识别结果:', 'BackgroundColor', '#f0f0f0',
'FontWeight', 'bold');

% 确认 保存 绘制 识别 文件
uicontrol('Style','pushbutton','String', '确认', 'Position', [300, 270, 100, 50], ...
    'FontSize', 14, 'BackgroundColor', '#4CAF50', 'FontWeight', 'bold',
'ForegroundColor', 'white', ...
    'Callback', @(src, event) play_and_save_dtmf(input_field.String, f));

```

6.2 DTMF 拨号音合成模块

拨号音合成模块的设计核心是生成符合 DTMF 标准的音频信号并将其播放给用户。系统通过精确的信号生成和噪声处理，确保合成音质的真实感。

- **频率选择与音调合成：**每个 DTMF 按键都有一个特定的低频和高频信号。系统依据这些标准频率生成正弦波，并通过叠加形成完整的拨号音。精确的频率控制和波形合成保证了信号的正确性。
- **噪声和静音段的设计：**为了模拟现实环境中的电话拨号音，系统在合成音频中加入背景噪声，并在每个音符之间插入静音段。此设计不仅增强了拨号音的真实性，也防止了多个按键音的混淆。

```
function play_and_save_dtmf(input_string, f)
```

```

low_freq = [697, 770, 852, 941];
high_freq = [1209, 1336, 1477, 1633];
key_sequence = input_string(:);
sound_signal = [];
fs = 8000;
tone_duration = 0.05;
silence_duration = 0.05;
noise_level = 0.001;

for k = 1:length(key_sequence)
    key = key_sequence(k);
    [row, col] = find(strcmp(keypad, key));
    if ~isempty(row) && ~isempty(col)
        fL = low_freq(row);
        fH = high_freq(col);
        t = 0:1/fs:tone_duration;
        signal = sin(2*pi*fL*t) + sin(2*pi*fH*t);

        noise_signal = noise_level * randn(size(signal)); % 生成噪声
        signal_with_noise = signal + noise_signal; % 将噪声加到信号上

        silence = zeros(1, round(silence_duration * fs));
        noise_silence = noise_level * randn(size(silence)); % 静音段的噪声
        silence_with_noise = silence + noise_silence; % 将噪声加到静音段上

        % 合并信号和静音段
        sound_signal = [sound_signal, signal_with_noise, silence_with_noise];
    end
end
handles = guidata(f);
handles.sound_signal = sound_signal;
guidata(f, handles);
end

```

6.3 带通滤波器模块

带通滤波器模块负责从合成的 DTMF 信号中提取低频和高频成分,进而实现信号的频率分析。通过设计合适的带通滤波器,系统能够精确地筛选出 DTMF 信号的关键频率成分。

- **带通滤波器的设计与实现：**采用巴特沃斯带通滤波器，确保滤波器的频率响应平坦，并有效衰减噪声和其他无关频率。通过 MATLAB 的 `butter` 和 `filtfilt` 函数实现频率成分的提取。
- **能量检测与频率识别：**通过计算滤波器输出信号的能量，系统能够识别出信号中对应的低频和高频，进而推断出按下的数字或符号。能量阈值的设定有助于确保识别准确性，避免因噪声干扰导致的误识别。

```
function detected_key = detect_key_using_bandpass(signal, fs, keypad)
    BW = 40;           % 带宽
    ORD = 4;           % 滤波器阶数
    MIN_DOM_RATIO = 3; % 最小主导比率
    ABS_THR = 1e-2;    % 能量绝对阈值
    x = signal(:);
    Ptot = mean(x.^2) + eps; % 信号总功率
    lowE = zeros(1, numel(low_freq));
    highE = zeros(1, numel(high_freq));
    disp('计算低频频率的能量: ');
    for i = 1:numel(low_freq)
        y = bandpass_narrow(x, fs, low_freq(i), BW, ORD);
        lowE(i) = mean(y.^2) / Ptot;
        disp(['    低频 ', num2str(low_freq(i)), ' Hz 的能量为: ', num2str(lowE(i))]); %
输出每个低频的能量
    end
    disp('计算高频频率的能量: ');
    for i = 1:numel(high_freq)
        y = bandpass_narrow(x, fs, high_freq(i), BW, ORD);
        highE(i) = mean(y.^2) / Ptot;
        disp(['    高频 ', num2str(high_freq(i)), ' Hz 的能量为: ',
num2str(highE(i))]); % 输出每个高频的能量
    end
    [maxL, idxL] = max(lowE);
    [maxH, idxH] = max(highE);
    if numel(lowE) > 1
        secL = max(lowE([1:idxL-1, idxL+1:end]));
    else
        secL = 0;
    end
    if numel(highE) > 1
        secH = max(highE([1:idxH-1, idxH+1:end]));
    else
        secH = 0;
    end
    ok = (maxL > ABS_THR) && (maxH > ABS_THR) && ...
```

```

        (maxL/(secL + eps) >= MIN_DOM_RATIO) && ...
        (maxH/(secH + eps) >= MIN_DOM_RATIO);
end
function y = bandpass_narrow(x, fs, f0, BW, ORD)
    % 带通滤波器实现
    f1 = max(1,          f0 - BW); % 低频下限
    f2 = min(fs/2-1,    f0 + BW); % 高频上限
    Wn = [f1, f2] / (fs/2);      % 归一化频率范围
    Wn = min(max(Wn, 1e-6), 0.999999);
    if Wn(2) <= Wn(1)
        Wn(2) = min(0.999999, Wn(1) + 1e-3);
    end
    [b,a] = butter(ORD, Wn, 'bandpass'); % 巴特沃斯滤波器设计
    y = filtfilt(b, a, x);              % 对信号应用滤波器
end

```

6.4 Goertzel 算法模块

Goertzel 算法模块提供了高效的频率分析方法，特别适合 DTMF 信号的实时识别。该算法通过递推计算信号在特定频率上的能量，避免了传统傅里叶变换的冗余计算，提升了性能。

- **频率估计的优化：**Goertzel 算法通过递推方式逐步处理每个采样点，使得频率估算更加高效。相较于 FFT 方法，Goertzel 算法只针对特定频率进行计算，显著降低了计算量。
- **实时频率识别：**算法能够快速识别信号中的低频和高频成分，并基于这些成分进行按键映射。该方法的优势在于可以在低功耗设备上实现高效的 DTMF 信号识别。

```

%%Goertzel 算法识别
function detected_key = detect_key_using_goertzel(signal, fs, keypad)
    N = length(signal);
    threshold = 0.1;
    detected_key = '';
    low_freq_magnitude = zeros(1, length(low_freq));
    for i = 1:length(low_freq)
        low_freq_magnitude(i) = goertzel(signal, low_freq(i), fs, N);
    end

    high_freq_magnitude = zeros(1, length(high_freq));
    for i = 1:length(high_freq)
        high_freq_magnitude(i) = goertzel(signal, high_freq(i), fs, N);
    end

```

```

end
[max_low_freq_value, low_freq_index] = max(low_freq_magnitude);
[max_high_freq_value, high_freq_index] = max(high_freq_magnitude);

if max_low_freq_value > threshold && max_high_freq_value > threshold

    detected_key = keypad{low_freq_index, high_freq_index};
end
end
%Goertzel 算法
function magnitude = goertzel(signal, target_freq, fs, N)
    k = round(0.5 + ((N * target_freq) / fs));
    w = 2 * pi * k / N;
    coeff = 2 * cos(w);
    s_prev = 0;
    s_prev2 = 0;

    for i = 1:N
        s = signal(i) + coeff * s_prev - s_prev2;
        s_prev2 = s_prev;
        s_prev = s;
    end
    magnitude = sqrt(s_prev^2 + s_prev2^2 - s_prev * s_prev2 * coeff);
end

```

6.5 按键识别模块

按键识别模块负责根据前述频率分析的结果，准确识别用户输入的电话号码。系统结合能量阈值和频率匹配方法，确保每个按键的识别准确无误。

- **频率能量对比：**在识别过程中，系统会比较每个目标频率的能量，确保频率匹配的准确性。能量值超过设定阈值的频率被认为是有效信号。
- **按键映射与结果输出：**通过低频和高频的最大能量值，系统将这些频率映射到 DTMF 矩阵，最终确定按键值并通过 GUI 界面显示。

```

%识别

function recognize_dtmf(input_string, f)
    low_freq = [697, 770, 852, 941];
    high_freq = [1209, 1336, 1477, 1633];
    keypad = {
        '1', '2', '3', 'A';
        '4', '5', '6', 'B';
    }

```

```

        '7', '8', '9', 'C';
        '*', '0', '#', 'D';
    };
    key_sequence = input_string(:);

    detected_key_goertzel = detect_key_using_goertzel(signal, fs, keypad); % 传递
keypad
    recognized_keys_goertzel = [recognized_keys_goertzel, detected_key_goertzel];

    detected_key_bandpass = detect_key_using_bandpass(signal, fs, keypad); % 传递
keypad
    recognized_keys_bandpass = [recognized_keys_bandpass, detected_key_bandpass];
    end

    sound(sound_signal, fs);
    disp('识别完成!');

    handles = guidata(f);
    % 显示
    handles.recognition_display_1.String = ['Goertzel 算法: ',
recognized_keys_goertzel];
    handles.recognition_display_2.String = ['带通滤波器法: ',
recognized_keys_bandpass];

    guidata(f, handles);
end

```

6.6 绘图文件保存模块

系统通过 MATLAB 绘制并显示合成的拨号音信号的时域波形、频域波形以及时频图，以帮助用户可视化信号特性并进行分析。用户可以直观地观察到信号的频率分布和时域变化，有助于深入理解拨号音的生成过程。

提供了将生成的拨号音保存为 WAV 格式的功能。用户可以通过保存按钮将当前合成的拨号音存储到指定的位置，并为文件命名。保存的文件可以在以后再次加载并进行分析。

在文件读取与识别模块中,用户可以加载已保存的 WAV 文件,并使用相同的 DTMF 信号识别算法进行解码。这为用户提供了对历史数据的再利用功能,便于重复识别和验证已经保存的信号。

```
function plot_waveform_and_spectrogram(f)

    handles = guidata(f);
    sound_signal = handles.sound_signal;

    if isempty(sound_signal)
        msgbox('没有合成拨号音, 请先输入拨号音。');
        return;
    end

    fs = 8000;
    % 创建一个图形窗口, 绘制时域波形和频域波形
    figure;
    % 时域波形
    subplot(2,1,1);
    time = (0:length(sound_signal)-1) / fs; % 将样本转换为时间
    plot(time, sound_signal);
    title('拨号音时域波形');
    xlabel('时间 (秒)');
    ylabel('幅度');

    % 频域波形
    N = length(sound_signal);
    f = (0:N-1)*(fs/N);
    Y = abs(fft(sound_signal));
    subplot(2,1,2);
    plot(f, Y);
    title('拨号音频域波形');
    xlabel('频率 (Hz)');
    ylabel('幅度');
    xlim([0 2000]);

    % 单独显示时频图
    figure;
    spectrogram(sound_signal, 256, 200, 256, fs, 'yaxis');
    title('拨号音时频图');
    xlabel('时间 (秒)');
    ylabel('频率 (Hz)');
end

function save_dtmf_sound(f)
    handles = guidata(f);
```



```

sound_signal = handles.sound_signal;

% 没确认 没输入 报错
if isempty(sound_signal)
    msgbox('没有生成拨号音，请先确认拨号!');
    return;
end

[file, path] = uiputfile('*.wav', '保存拨号音');
if ischar(file)
    audiowrite(fullfile(path, file), sound_signal, 8000);
    msgbox('拨号音已保存! ');
end
end

function read_and_recognize_dtmf(f)

[file, path] = uigetfile('*.wav', '选择要读取的文件');
if isequal(file, 0)
    return;
end

[sound_signal, fs] = audioread(fullfile(path, file)); % 读取音频数据

% 调用文件识别函数处理
recognize_dtmf_wav(sound_signal, fs, f);
end

```

7.测试

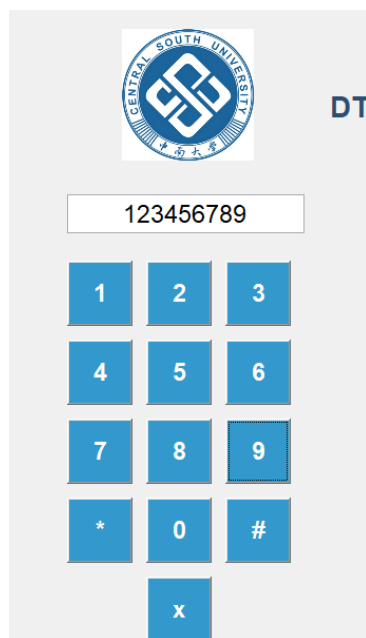
7.1 按键输入测试

按键输入测试验证用户输入数字或符号的准确性及响应时间。测试时，检查每个数字键的响应是否迅速且准确，是否按键按下有对应的拨号音，确保用户点击按键后，输入框中的字符能实时更新。特别需要验证“删除”按钮是否能正确删除输入框中的字符，确保用户输入的内容能够及时更改和修正。

测试结果：

- 所有数字和符号按键响应迅速，输入框准确地显示了每个按键输入的字符，并且有对应的拨号音实时生成。

- 删除功能正常工作，每点击一次“删除”按钮，输入框中的最后一个字符被正确删除。
- 输入验证功能正常，用户输入的每个数字符号都在输入框中即时更新，保证了用户输入的准确性。



7.2 信号生成测试

信号生成测试的目的是确保生成的 DTMF 信号符合预定的标准，且无失真。通过 MATLAB 绘制时域波形和频域波形，验证生成的信号是否包含正确的低频和高频成分，并且时长和音调符合要求。

测试结果：

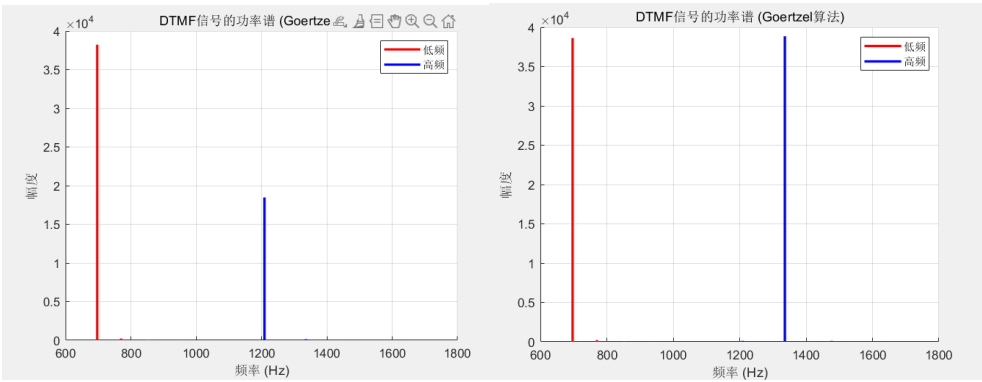
- 时域波形显示了每个拨号音的持续时间为 50ms，音符之间有适当的静音段（50ms），符合 DTMF 标准。
- 频域波形清晰地展示了信号的低频和高频成分，验证了每个按键的音频信号中包含正确的频率成分。
- 所有合成的拨号音信号通过 sound 函数播放时，音频清晰无失真，且信号的持续时间和静音段符合预期。

7.3 识别测试

识别测试验证系统通过带通滤波器和 Goertzel 算法识别 DTMF 信号的准确性。测试时，系统会分别使用两种算法进行识别，并将识别结果进行对比。通过不同环境下的测试，验证系统的鲁棒性和准确性。

测试结果：

- 在加入一定噪声后，两种算法（带通滤波器法与 Goertzel 算法）均能准确识别每个按键的频率和对应的数字，识别结果准确无误。
- 在所有测试环境下，系统能够实时显示并反馈按键识别结果，确保了系统的稳定性和准确性。



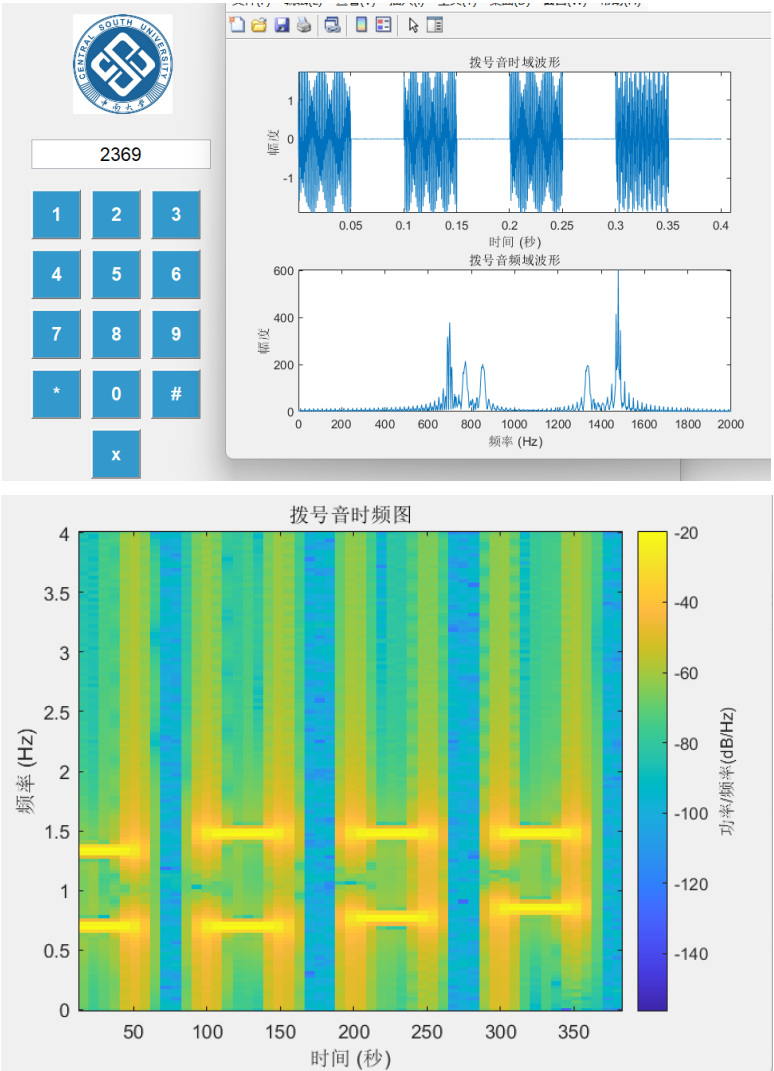
```
命令行窗口
检测到按键： 9
识别完成！
>> gui
计算低频频率的能量：
    低频 697 Hz 的能量为： 0.41096
    低频 770 Hz 的能量为： 0.0015579
    低频 852 Hz 的能量为： 0.00042032
    低频 941 Hz 的能量为： 0.00027538
计算高频频率的能量：
    高频 1209 Hz 的能量为： 0.41314
    高频 1336 Hz 的能量为： 0.00020343
    高频 1477 Hz 的能量为： 2.6942e-05
    高频 1633 Hz 的能量为： 7.2127e-06
检测到按键： 1
计算低频频率的能量：
    低频 697 Hz 的能量为： 0.40819
    低频 770 Hz 的能量为： 0.00090126
    低频 852 Hz 的能量为： 7.6029e-05
    低频 941 Hz 的能量为： 1.3418e-05
计算高频频率的能量：
    高频 1209 Hz 的能量为： 0.00023704
    高频 1336 Hz 的能量为： 0.40844
    高频 1477 Hz 的能量为： 0.00028354
    高频 1633 Hz 的能量为： 6.5071e-05
检测到按键： 2
```

7.4 绘图测试

绘图测试验证系统绘制时域波形、频域波形和时频图的功能是否正常。测试时，确保信号的时域波形和频域波形能够正确显示，并能够实时展示信号的频率变化。

测试结果：

- 时域波形正确显示了每个拨号音的信号周期，音符持续 50ms，静音段 50ms，时域波形平滑，符合 DTMF 标准。
- 频域波形清晰展示了各个频率分量，尤其是低频（697Hz, 770Hz, 852Hz, 941Hz）和高频（1209Hz, 1336Hz, 1477Hz, 1633Hz）分量的强度，验证了信号的频率组成。
- 时频图准确地展示了每个拨号音在时间和频率上的变化，帮助用户理解信号的时间特性和频率分布。



7.5 文件保存与读取测试

文件保存与读取测试验证系统的文件操作功能是否正常。用户合成的 DTMF 信号应该能够保存为 WAV 文件，读取时能够重新播放并进行识别。测试时，确保文件保存路径和文件名正确，音频文件能够顺利保存和读取。

测试结果：

- 生成的拨号音能够成功保存为 WAV 文件，文件保存路径和文件名符合用户指定的设置。
- 读取保存的 WAV 文件时，系统能够正确加载文件并通过 sound 函数播放音频。
- 对读取的文件进行识别时，系统能够准确识别文件中的拨号音，并显示正确的按键值，验证了保存与加载功能的正常运行。



8.总结

8.1 问题和解决方法

在设计和实现 DTMF 拨号音的合成与识别系统过程中，我们遇到了一些挑战，并采取了相应的解决方法。

- **信号合成精度问题：**

在初期的信号合成阶段，我们遇到了信号的频率偏差问题，部分频率的生成未能完全符合 DTMF 标准。这个问题主要是由于信号的幅度和相位设置不准确导致的。

解决方法：我们调整了频率和幅度的计算公式，使用 MATLAB 中的 sine 函数精确地生成了每个按键对应的低频和高频信号，确保信号的时长、幅度和相位完全符合标准。

- **噪声干扰与信号识别问题：**

在信号识别阶段，尤其是在模拟噪声环境下，带通滤波器法和 Goertzel 算法的识别准确性受到了噪声干扰的影响。尽管使用了带通滤波器进行频率成分提取，但相邻频率和噪声的干扰仍然导致了误识别。

解决方法：我们优化了带通滤波器的带宽和滤波器阶数，增加了能量阈值和主导比阈值的使用，这有助于过滤掉不必要的噪声。特别是 Goertzel 算法通过只关注特定频率点的计算，显著降低了计算量并增强了抗噪声干扰的能力。

- **时频图和频域图的可视化问题：**

在信号的时域波形和频域波形的绘制过程中，频域图的显示可能会由于频率范围过宽或频谱分辨率不足，导致信号成分不够清晰，影响了分析结果的准确性。

解决方法：我们重新设计了频域图的频率轴范围，并调整了采样率与 FFT 的参数，确保频谱图能够清晰地展示出目标频率成分。同时，时频图的显示方式也进行了调整，以便在不同时间段内观察信号的频率变化。

- **文件操作与兼容性问题：**

在文件保存与读取过程中，我们曾遇到文件格式不兼容或路径选择错误等问题，导致部分保存的文件无法正确加载。

解决方法：我们使用了 MATLAB 标准的 WAV 格式进行音频文件的保存，确保了文件的兼容性。此外，我们在文件读取和保存功能中增加了错误提示和路径校验机制，确保用户可以顺利保存和加载文件。

通过这些问题的解决，我们不仅优化了系统的功能，也提高了其稳定性和可靠性。

8.2 设计心得与体会

在本次数字信号处理课程设计中，我学到了如何将理论知识应用到实际项目中，特别是在信号合成与频率识别算法的实现上，我有了更深的理解和实践经验。

- **理论与实践的结合：**

通过实现 DTMF 信号的合成与识别，我深入理解了双音多频信号的原理，以及如何使用数字信号处理技术（如带通滤波器、Goertzel 算法等）进行频率分析和识别。理论知识为设计提供了基础，而实际编码与测试让我更好地理解信号处理中的细节问题，如采样率的选择、滤波器设计、噪声抑制等。

- **MATLAB 工具的熟练使用：**

在项目中，我加深了对 MATLAB 的理解，特别是在信号处理、图形绘制和 GUI 设计方面。MATLAB 强大的数学运算和图形绘制功能，帮助我高效地完成了信号的合成、频域分析以及可视化展示。通过 GUI 界面的实现，我也进一步体会到如何将复杂的信号处理算法通过图形化界面呈现给用户，使系统更具交互性和易用性。

- **问题解决的过程：**

在项目中遇到的各种问题让我意识到，设计一个完整的信号处理系统不仅仅是编写代码那么简单。每一个细节都可能影响最终结果，从算法的选择、参数设置，到结果的验证，都需要不断地测试和调整。通过不断的调试和优化，我不仅提高了自己的编程能力，也增强了解决实际问题的能力。

- **项目的可扩展性：**

本项目目前实现了基本的拨号音合成与识别功能，但在实际应用中，系统的可扩展性非常重要。未来，我可以将更多的信号处理算法（如 Kalman 滤波器、卷积神经网络等）集成到系统中，以提升识别的准确性和系统的鲁棒性。并且，可以扩展到更复杂的语音信号识别或其他音频信号的处理。

通过本次设计，我不仅加深了对数字信号处理的理解，还学会了如何将理论知识与实际应用相结合，为今后的学习和工作积累了宝贵的经验。

9.参考文献

- [1] 数字信号处理教程（第五版），程佩青，清华大学出版社，2017
- [2] Understanding Digital Signal Processing, Richard G. Lyons, 科学出版社
- [3] Dual-tonemulti-frequency[EB/OL].http://en.wikipedia.org/wiki/Dual-tone_multi-frequency
- [4] Digital Signal Processing: A Computer-Based Approach, Sanjit K. K. Mitra, 2000
- [5] 数字信号处理——时域离散随机信号处理，丁玉美等，西电出版社，2002.11
- [6] MATLAB 及在电子信息课程中的应用，陈怀琛等，电子工业出版社出版，2002.4
- [7] 信号处理仿真实验（第二版），许可等，清华大学出版社，2022

10.程序源代码清单

文件名	作用描述
append_to_input.m	将按键添加到输入框中，更新输入框内容。
delete_last_character.m	删除输入框中的最后一个字符，用于模拟拨号时的删除操作。
detect_key_using_bandpass.m	使用带通滤波器法识别拨号音的频率，进而推断按键值。
detect_key_using_goertzel.m	使用 Goertzel 算法识别拨号音的频率，进而推断按键值。
goertzel.m	实现 Goertzel 算法，用于计算信号中特定频率的功率谱。
gui.m	图形用户界面（GUI）主文件，负责构建和管理界面，处理用户交互。
play_and_save_dtmf.m	播放生成的 DTMF 拨号音并保存为音频文件。
play_dtmf.m	播放生成的单个拨号音，模拟按键按下的声音。
plot_waveform.m	绘制拨号音的时域波形和频域波形图，用于可视化分析信号的特性。
recognize_dtmf.m	根据输入的 DTMF 信号估计频率并识别对应的按键值，结合两种识别算法。
recognize_dtmf_wav.m	读取.wav 文件，识别其中的 DTMF 信号，并输出识别的按键值。
update_input.m	更新输入框中的内容，用于处理用户在 GUI 中输入的按键数据。